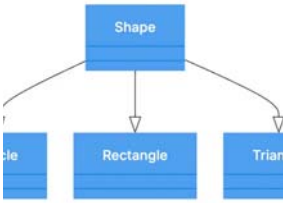
B 3.2.1 Inhertitance (HL Only)



B3.2.1 Explain and apply the concept of inheritance in OOP to promote code reusability. (HL Only)

This section explores inheritance in OOP for code reusability. It explains hierarchical relationships between parent and child classes, how to extend classes, and the impact of access modifiers (private, public, protected, default) on inherited members.

Learning Objectives Java

- Students will be able to explain the concept of inheritance in OOP and its role in promoting code reusability.
- Students will be able to identify superclasses (parent) and subclasses (child) and describe the hierarchical relationship between them.
- Students will be able to extend existing Java classes using the `extends` keyword to reuse and extend functionalities.
- Students will be able to explain the impact of different access modifiers (`private`, `public`, `protected`, default) on the accessibility of superclass members in subclasses.

Relevant ATL Skills:

- **Thinking Skills:** Understanding concepts, applying knowledge to new scenarios, analysing code structure.
- **Communication Skills:** Articulating the principles of inheritance, discussing code examples.

Learning Objectives Python

- Students will be able to explain the concept of inheritance in OOP and its role in promoting code reusability.
- Students will be able to identify parent (base) and child (derived) classes and describe the hierarchical relationship between them.
- Students will be able to extend existing Python classes through inheritance to reuse and extend functionalities.

- Students will be able to explain how Python's conventions for access control (name mangling and naming conventions) impact the accessibility of parent class members in child classes.

Relevant ATL Skills:

- **Thinking Skills:** Understanding concepts, applying knowledge to new scenarios, analysing code structure.
- **Communication Skills:** Articulating the principles of inheritance, discussing code examples.

Lesson Resources

Presentation Java

Link to presentation

Presentation Python

Link to presentation

eBook

B 3.2.1 Inheritance in OOP Java (HL Only)

B 3.2.1 Inheritance in OOP Python (HL Only)

Other Resources

B 3.2.1 Worksheet Java

B 3.2.1 Worksheet Java Answers

B 3.2.1 Worksheet Python

B 3.2.1 Worksheet Python Answers

Lesson Planning

Lesson Plan Java

Time (minutes)	Activity	Content Covered	Slide(s)
0-5	Introduction & Learning Objectives	Recap of OOP fundamentals (classes, objects), introduction to inheritance, lesson objectives	1-2
5-15	What is Inheritance?	Definition, benefits (code reusability, maintainability), "is-a" relationship, superclass and subclass	3-5
15-25	Inheritance in Java	Syntax (`class Subclass extends Superclass`), the `super()` keyword for calling superclass constructors	6-8
25-35	Method Overriding & Extension	Overriding superclass methods (`@Override` annotation), extending functionality by adding new methods in the subclass	9-11
35-45	Access Modifiers and Inheritance	Impact of `public`, `private`, `protected`, and default (package-private) access modifiers on inherited members	12-15
45-55	Practical Example & Discussion	Live coding or demonstration of a simple inheritance example (e.g., `Animal` and `Dog` classes), class discussion on the benefits and implications	16-17
55-60	Wrap-up & Worksheet Introduction	Review of key concepts, introduction to the worksheet for self-assessment	18

Lesson Plan Python:

Time (minutes)	Activity	Content Covered	Slide(s)
0-5	Introduction & Learning Objectives	Recap of OOP fundamentals (classes, objects), introduction to inheritance, lesson objectives	1-2
5-15	What is Inheritance?	Definition, benefits (code reusability, maintainability), "is-a" relationship, parent and child classes	3-5
15-25	Inheritance in Python	Syntax (`class Child(Parent):`), the `__init__` method in child classes, calling the parent's `__init__` using `super()`	6-8
25-35	Method Overriding & Extension	Overriding parent methods, extending functionality by adding new methods in the child class	9-11

Time (minutes)	Activity	Content Covered	Slide(s)
35-45	Access Control in Python	Python's convention for "private" members (name mangling with double underscore __), protected and public members (by convention), accessibility in child classes	12-14
45-55	Practical Example & Discussion	Live coding or demonstration of a simple inheritance example (e.g., `Animal` and `Dog` classes), class discussion on the benefits and implications	15-16
55-60	Wrap-up & Worksheet Introduction	Review of key concepts, introduction to the worksheet for self-assessment	17

Teacher Notes

Teacher Notes (Java):

- **Emphasise the "Is-A" Relationship:** Similar to Python, ensure students understand when inheritance is appropriate. Use the "is-a" test.
- **`super()` in Java:** Stress that the call to `super()` must be the first statement in the subclass constructor. Explain why (superclass initialisation needs to happen before subclass-specific initialisation).
- **Access Modifiers are Crucial:** Spend adequate time explaining the nuances of each access modifier and how they affect inheritance. Use diagrams if necessary to illustrate visibility. Common misunderstandings involve the accessibility of `private` members and the scope of `protected`.
- **`@Override` Annotation:** Emphasise the importance of using `@Override`. Explain that it's not strictly necessary for the code to run, but it's good practice for catching errors and improving code readability.
- **Practical Examples:** The live coding should clearly demonstrate the effects of different access modifiers on inherited members. Show examples of trying to access `private` members from the subclass and the resulting compilation errors.
- **Abstract Classes and Interfaces (Future Connection):** Briefly mention that inheritance is a key concept that leads to more advanced OOP topics like abstract classes and interfaces, which enforce certain structures in subclasses.

Teacher Notes (Python):

- **Emphasise the "Is-A" Relationship:** Students sometimes struggle to identify appropriate inheritance hierarchies. Use real-world examples and stress the "is-a" test (e.g., a square *is a* type of rectangle - might lead to issues; a dog *is a* type of animal - makes more sense).
- **`super()` is Key:** Ensure students understand the importance of using `super()` to initialise parent class attributes. Forgetting this is a common error. Explain what happens if `super().__init__()` is not called (parent's `__init__` is not executed).
- **Python's Access Conventions:** Be clear that Python's access control is based on convention, not strict enforcement. Explain the purpose of name mangling (to avoid accidental name clashes in subclasses) but that it doesn't provide true privacy. Compare this to languages with explicit access modifiers if students have prior programming experience.
- **Practical Examples:** The live coding or demonstration should be interactive. Encourage students to suggest classes and attributes/methods for the hierarchy. Start with simple examples and gradually increase complexity.
- **Method Overriding vs. Overloading:** If students have encountered other programming languages, they might confuse overriding with overloading (which Python handles differently). Clarify the distinction: overriding is about providing a new implementation of an *existing* method in a subclass.